Business Requirements Document (BRD)

TaskFlow AI Support Agent

Document Version	2.0
Date	2026-08-24
Author	Vansh Rautela
Supersedes	BRD v1.0 (2026-08-24)
Companion	`PRODUCT_SPECIFICATION.md`

```
Status:        v1 implemented — core pipeline complete, delivery/SLA hardening in progress
Last verified: 2026-08-24 via the commands below
Build health:  lint × (24 ruff findings) · types × (9 mypy errors in 5 files)
               · import-boundaries ✓ (1 contract kept, 0 broken) · tests ✓ (68/68 passing)
Verified by:   `uv run pytest -q` (68 passed in 38.32s)
               `uv run lint-imports` (102 files, 222 dependencies analysed)
               `uv run ruff check .` · `uv run mypy src/` · `uv run alembic heads` (d300852af25a)
```

Test layers: unit (64 tests / 12 files) and pipeline (2) and integration (2) are complete and green. `tests/e2e/` exists but is **empty** — end-to-end scenario tests are the next test-layer deliverable, not a claimed capability. Lint and type-check findings are concentrated in the adapter and script layers (`adapters/channels/` , `api/app.py` , `dashboard/app.py`); the `domain/` and `services/` layers that the architecture contract protects are clean. See §13 and §14 for the evidence trail.

---

1. Executive Summary

TaskFlow is an AI customer-support agent that takes an inbound customer message, decides what it is about, retrieves the supporting knowledge-base passages, drafts a reply that cites them, and then decides — deterministically and on the record — whether that reply is safe to send without a human reading it first.

The engineering thesis is not "an LLM writes a support reply." It is:

> **A system that knows when it is not allowed to answer, and can prove why.**

Every architectural choice serves that sentence. Safety checks are **conjunctive veto gates evaluated before** any quality arithmetic, so a critical policy violation can never be averaged away by strong scores elsewhere. Seven such gates run on every message. Only drafts that clear all seven reach the weighted quality score, which decides among survivors. Every decision carries a single reason code and a complete trace that can be reconstructed months later from its `trace_id` alone.

The system is built around a fictional B2B SaaS company, **TaskFlow**, used as the demo vehicle for this work (see §3.2). It ingests through a REST API and a Gmail IMAP poller, classifies into 8 intents, retrieves over a 42-document knowledge base using hybrid dense and BM25 search with reciprocal-rank fusion, drafts with a three-provider failover chain (OpenRouter → Claude → Ollama) that degrades to a fully local model with no network, runs three concurrent validators fail-closed, evaluates the seven gates, scores five weighted quality signals, and then either enqueues the reply to a transactional outbox or creates a human-review item with a multi-channel alert.

What exists today is a complete, runnable governance pipeline with 68 passing tests and a CI-enforced architectural boundary. What is deliberately deferred — reranking, a trained classifier, spam filtering, the FAQ fast path, outbox retry semantics, SLA sweeping — is listed as roadmap in §7.3, with the code-level status of each stated plainly rather than implied.

---

2. Problem Statement

2.1 The problem is governance, not generation

Generating a plausible support reply with a modern LLM is close to a solved problem. The unsolved problem — the one that keeps AI support out of production at companies with real compliance exposure — is knowing when the generated reply must not be sent, and being able to demonstrate afterwards why a given reply was or was not sent.

Four failure modes make an ungoverned support agent unshippable:

1. **Unauthorised commitments.** A model that has read a refund policy will happily approve a $750 refund when the agent ceiling is $500, promise a ship date for an unreleased feature, or invent a discount. These are business commitments made by a system with no authority to make them.
2. **Confident hallucination.** A model asked a question the knowledge base does not answer will answer anyway. Without a sufficiency check and a grounding measurement, the system has no way to distinguish "answered from the corpus" from "answered from nowhere."
3. **Unexplainable decisions.** "The confidence was 0.83" is not an audit trail. When a customer disputes a reply three months later, the operator needs the intent, the exact chunks retrieved, the draft, every validator result, every gate result, the weights, the score, the threshold, and the provider and cost of each model call.
4. **Silent safety degradation.** If a safety signal is one weighted term among many, a strong retrieval score can numerically compensate for a policy violation. The system then sends an unsafe reply while reporting high confidence.

2.2 The design response

TaskFlow answers each failure mode with a specific, testable mechanism:

Failure mode	Mechanism	Where it lives
Unauthorised commitments	10 configurable policy rules; 7 critical, evaluated against both inbound and outbound text; any critical hit vetoes auto-send via Gate G1	`config/policies.yaml`, `src/taskflow/services/policy/service.py`
Confident hallucination	Retrieval sufficiency check (top score, support count) plus a grounding measurement over the retrieved context, plus citation resolution against real chunk IDs	`src/taskflow/services/retrieve/sufficiency.py`, `src/taskflow/services/validate/`
Unexplainable decisions	A `Trace` record per message capturing every stage, plus a single reason code per decision, queryable over REST	`src/taskflow/domain/models.py`, `src/taskflow/api/app.py`
Silent safety degradation	Two-layer confidence: conjunctive gates evaluated **before** the weighted score; safety is never a weight	`src/taskflow/services/confidence/gates.py`, `scorer.py`

The regression guard for the fourth point is `tests/unit/test_gates.py::test_critical_policy_violation_never_auto_sends`, which constructs a draft with **every quality signal at 1.0** — a score above 0.95, comfortably over any threshold — and asserts it still routes to human review because a critical policy violation is a veto, not a number. That test is the centrepiece of the design.

---

3. Business Context

3.1 The real business driver

The genuine driver for this project is an **internship capstone presented to senior engineers and engineering leadership**. It is a solo build, four weeks, free-tier infrastructure only, on a machine with no Docker. The document you are reading is written for that audience: engineers who will read the code after they read this and will notice any gap between the two.

That context shapes the requirements in ways worth stating up front:

- **Explainability outranks throughput.** The system must be explained live, component by component, by the one person who wrote all of it. That ruled out an orchestration framework in favour of a ~220-line explicit async pipeline (ADR-005).
- **The demo must not fail.** A dependency that can go down mid-presentation is a design risk. That is why `TASKFLOW_LLM_MODE=local_only` exists, why Qdrant is embedded rather than containerised (ADR-007), and why the queue is a SQLite table rather than Redis (ADR-006).
- **Honest measurement outranks flattering measurement.** Where a capability is configured but not wired in, this document says so rather than describing the config as behaviour.

3.2 TaskFlow is the demo vehicle, not a customer

TaskFlow is a fictional B2B SaaS company invented for this project. It sells project-management software on three tiers — Free, Pro at $12/user/month, Enterprise at $29/user/month. Every customer message, every knowledge-base article, and every evaluation record in this repository is synthetic. There is one tenant (`taskflow-demo`). There are no real users, no revenue, and no production traffic.

Consequently this document contains **no revenue projections, no ROI percentages, no customer counts, and no market sizing.** Inventing those numbers would make the document less credible, not more. The business objectives in §6 are stated instead as **capability targets** — auto-send rate, zero unsafe sends, cost ceiling, latency — because those are real properties of the system, measurable from the code and the configuration, and falsifiable by a reviewer with a terminal.

3.3 Why this project shape

A support agent was chosen over the more common "chatbot over documents" demo because it forces the interesting constraints to be real rather than decorative:

- Support has **genuine policy surface** (refund ceilings, credential requests, legal threats, GDPR deletion), so safety rules have concrete content instead of being generic content-moderation gestures.
- Support has a **natural human-in-the-loop** — the correct output for a hard case is an escalation, not a confident guess — which makes the review queue, the SLA, and the approve/edit/reject workflow load-bearing rather than ornamental.
- Support has an **exactly-once delivery requirement**, which makes the transactional outbox and idempotency key necessary rather than performative.

---

4. Stakeholders

These are the TaskFlow personas the system is designed around. They are fictional roles in a fictional company, but the interaction surfaces they describe are real and implemented.

Role	What they need	How they interact with TaskFlow	Implemented surface
Customer	A correct, fast, policy-consistent answer — or a fast handoff to a person	Sends email to the support address, or a message via `POST /messages`	`adapters/channels/gmail_poller.py`, `api/app.py`
Support Operator	To see the customer's message and the AI draft side by side, with the reason it was escalated, and to approve, edit, reject, or escalate	Streamlit operator console, tab 1	`dashboard/app.py`, `services/review/service.py`
Support Manager	Auto-send rate, human-review rate, LLM spend, and the ability to move thresholds without a deploy	Streamlit console tab 3; edits `config/thresholds.yaml`	`dashboard/app.py`, `GET /metrics`
Engineering	To extend the knowledge base, tune retrieval, and be certain the architecture has not rotted	`data/knowledge_base/`, `config/`, `scripts/`, CI boundary check	`scripts/ingest_kb.py`, `uv run lint-imports`
Compliance / Legal	To define what the agent may never say, and to audit any past decision	Edits `config/policies.yaml`; reads traces by `trace_id`	`services/policy/service.py`, `GET /traces/{trace_id}`

5. Scope

5.1 In scope (implemented and verified)

Area	What is actually in the build
Inbound ingestion	REST `POST /messages`; Gmail IMAP polling daemon (`scripts/start_gmail_poller.py`)
PII redaction	Regex redaction of card numbers, SSNs, and `sk_` / `pk_` /Bearer secrets before classification
Intent classification	8 intents via LLM structured output, with a deterministic keyword classifier as fallback
Knowledge retrieval	Hybrid dense (BAAI/bge-small-en-v1.5, 384-dim cosine) + sparse (Qdrant/bm25, IDF) with server-side RRF fusion over 42 documents / 178 chunks; alias query expansion; sufficiency evaluation; prompt-injection detection
Response drafting	LLM structured output to the `DraftOutput` contract with inline `[chunk_id]` citations; safe template fallback when every provider fails
Validation	3 validators (PII leak, citation resolution, grounding) run concurrently, fail-closed on crash or timeout
Policy engine	10 rules from `config/policies.yaml` — 7 critical, 3 warning — over inbound and outbound text
Safety gates	7 conjunctive veto gates evaluated before scoring
Quality scoring	5 weighted signals summing to 1.0, compared against a configurable threshold
Human review	Persistent review queue with optimistic-locking state transitions, edit-diff recording, and a Streamlit operator console
Outbound delivery	Transactional outbox with a UNIQUE idempotency key and an atomic claim; background delivery worker over Gmail SMTP
Alerting	Independent Email, Telegram, and generic-Webhook alert dispatch on every escalation
Observability	Full per-message trace with per-call LLM cost, provider, model, tokens, and latency; REST trace and metrics endpoints
LLM providers	OpenRouter → Claude → Ollama priority chain with automatic failover; `local_only` mode for zero-network operation

Area	What is actually in the build
Persistence	SQLite in WAL mode via async SQLAlchemy; Alembic migrations (head `d300852af25a`)

5.2 Out of scope for v1

Area	Rationale
Multi-tenant deployment	`tenant_id` columns and retrieval filters exist throughout; per-tenant keys, corpora, and queues are not built. Single tenant `taskflow-demo`
Phone / voice channels	Text only
Non-English messages	English only; no translation layer
Customer self-service portal	The product is operator-facing; customers interact only by message
Containerised or hosted deployment	Free-tier, single-machine, no Docker (ADR-007)
Load and soak testing	Design point is roughly one message every few seconds
Real PII/NER models	Regex redaction only; deliberate simplification, stated as such

5.3 Deferred — the v2 roadmap

These are decisions taken with a stated trigger for revisiting, not omissions. Each has partial scaffolding in the repository, which is why it is listed here rather than in §5.2.

Item	Current state in repo	Trigger to build it
Cross-encoder reranking	`rerank_k: 5` in config, `rerank_score` field on `ScoredChunk`; no reranker implemented. Retrieval is RRF top-5	Retrieval evaluation shows RRF top-5 missing the correct chunk
Trained intent classifier	`src/taskflow/ml/` is an empty package; `scikit-learn` / `joblib` are declared but unused; labelled datasets exist (35 train / 14 val / 14 test)	LLM classification cost or latency becomes the bottleneck, or offline accuracy must be certified
Spam filter	`spam:` block in `config/settings.yaml`; `RouteAction.REJECTED_SPAM` defined; no implementation	Inbound volume includes real spam
FAQ fast path	`fastpath:` block in config; `RouteAction.TEMPLATE_SENT` defined; no implementation	Repeat-question rate justifies a zero-LLM-call path
Pipeline-level deduplication	`try_claim` implemented and tested at the repository layer; not called by the pipeline; `RouteAction.DROPPED_DUPLICATE` never emitted	A poller can redeliver — this becomes required the moment the Gmail poller runs continuously
Outbox retry, lease, and DEAD state	`claim_lease_minutes`, `max_attempts` in config; no lease column, no retry counter increment, no requeue	First real delivery failure
SLA sweeper	`sla_minutes` in config; `ReviewRepository.due()` implemented but never called; deadline hardcoded to 4 hours	Operators need breach alerts rather than a static deadline
Per-intent thresholds in the pipeline	All eight thresholds are in `config/thresholds.yaml`; the orchestrator applies only `general` (0.70)	Immediately — this is the highest-value item on the list (see FR-8.3)
Web chat channel	`Channel.WEBCHAT` exists in the enum; no endpoint or page	A second inbound channel is needed

Item	Current state in repo	Trigger to build it
Telegram as an inbound channel	`adapters/channels/telegram.py` exists but does not import (see §15, R-7)	Two-way Telegram support is wanted

6. Business Objectives

Stated as capability targets, each traceable to configuration or code. No revenue, ROI, or market figures appear here by design (see §3.2).

#	Objective	Success metric	Target	Traceable to
BO-1	**Never auto-send an unsafe reply**	Auto-sent messages that violate any safety gate	**0** — absolute, non-negotiable	`services/confidence/gates.py` ; `tests/unit/test_gates.py` (6 tests)
BO-2	Every routing decision is explainable from its trace alone	% of decisions carrying a single machine-readable reason code and a full trace	100%	`RoutingDecision.reason_code` ; `GET /traces/{trace_id}`
BO-3	Auto-send a useful share of routine traffic	% of messages routed `auto_send` on non-preempt intents	≥ 60% target; measured, not asserted — see note below	`GET /metrics` → `auto_send_rate_pct`
BO-4	Survive a total cloud-LLM outage	System remains operational with no network	Full pipeline runs in `local_only` mode	`config/providers.yaml` `priority.local_only` ; keyword and template fallbacks
BO-5	Keep model spend bounded and visible	Per-call USD cost recorded; daily spend ceiling	≤ $2.00/day configured; **not yet enforced in code** (see NFR-5.2)	`config/providers.yaml` `budgets` ; `services/cost/service.py`
BO-6	First response fast enough to matter	Median time from ingestion to auto-send decision	< 10 s cloud path (P95); < 30 s local path (P95)	See NFR-1; not yet instrumented as a percentile
BO-7	Escalations reach a human immediately	Alert channels dispatched per escalation, each independently	3 channels, independent failure	`services/alert/service.py` ; `tests/unit/test_alert.py`

> **On BO-3.** The auto-send rate is a *measured output*, not a tuned input. Because the orchestrator currently applies the `general` threshold (0.70) to every intent rather than the per-intent thresholds in config, the observed rate today is higher than the design intends for `billing` (0.80) and `refund` (0.90). Wiring per-intent thresholds will *lower* the auto-send rate, which is the correct direction (ADR-001). The number should be re-measured after that change; quoting a rate now would be quoting an artefact.

7. Functional Requirements

Numbering follows the v1 draft where it still holds. Priorities: **P0** = must exist for the system to be defensible; **P1** = important, degradable.

FR-1 — Message ingestion

ID	Requirement	Priority	Status
FR-1.1	The system SHALL accept inbound messages via `POST /messages` with `message_id`, `tenant_id`, `channel`, `sender`, `subject`, `body_text`.	P0	Implemented
FR-1.2	The system SHALL support the channel types `email`, `webchat`, `console` in its domain model.	P0	Implemented (enum); `webchat` has no transport
FR-1.3	The system SHALL poll a Gmail mailbox over IMAP and run each unseen message through the pipeline.	P1	Implemented
FR-1.4	The system SHALL construct a deduplication key for every inbound message and persist it under a UNIQUE constraint.	P0	Implemented at the repository layer; **not invoked by the pipeline**
FR-1.5	Duplicate messages SHALL be dropped with a `DROPPED_DUPLICATE` routing decision.	P1	**Deferred** — action defined, never emitted

FR-2 — PII redaction

ID	Requirement	Priority	Status
FR-2.1	The system SHALL redact 13–16 digit card-number sequences from inbound text.	P0	Implemented
FR-2.2	The system SHALL redact US Social Security Numbers in `NNN-NN-NNNN` form.	P0	Implemented
FR-2.3	The system SHALL redact `sk_`/`pk_` secret keys (≥16 chars) and `Bearer` tokens (≥20 chars).	P0	Implemented
FR-2.4	Redactions SHALL use descriptive placeholders: `[REDACTED_CREDIT_CARD]`, `[REDACTED_SSN]`, `[REDACTED_API_KEY]`.	P1	Implemented
FR-2.5	The original text SHALL be preserved for the operator console; the redacted text SHALL be what reaches the LLM.	P0	**Partially implemented.** Classification receives redacted text; the drafting call passes `msg.body_text` unredacted (§15, R-1)

FR-3 — Intent classification

ID	Requirement	Priority	Status
FR-3.1	The system SHALL classify each message into exactly one of **8** intents: `billing`, `technical`, `account`, `feature_request`, `refund`, `complaint`, `general`, `unknown`.	P0	Implemented
FR-3.2	Classification SHALL use LLM structured output conforming to `ClassificationOutput` (`intent`, `confidence`, `reasoning`).	P0	Implemented
FR-3.3	If the LLM path fails for any reason, the system SHALL fall back to a deterministic keyword classifier rather than failing the message.	P0	Implemented
FR-3.4	The classifier SHALL return a confidence in [0.0, 1.0].	P0	Implemented (Pydantic-constrained)
FR-3.5	Empty or whitespace-only messages SHALL classify as `unknown` with confidence 0.0.	P1	Implemented

ID	Requirement	Priority	Status
FR-3.6	The classifier prompt SHALL enumerate exactly the intents in the `Intent` enum.	P0	**Not met.** The prompt lists 10 intents including two that do not exist; the extra labels fail validation and silently degrade to keyword fallback (§15, R-2)

FR-4 — Knowledge retrieval

ID	Requirement	Priority	Status
FR-4.1	Retrieval SHALL combine dense semantic search (`BAAI/bge-small-en-v1.5`, 384-dim, cosine) with sparse lexical search (`Qdrant/bm25`, IDF modifier).	P0	Implemented
FR-4.2	Dense and sparse result lists SHALL be combined by Reciprocal Rank Fusion.	P0	Implemented (Qdrant server-side `FusionQuery`)
FR-4.3	Queries SHALL be expanded using configured alias mappings (`2fa`, `sso`, `api key`, `invoice`).	P1	Implemented
FR-4.4	The system SHALL judge retrieval sufficiency using top score ≥ 0.35 and ≥ 2 supporting chunks scoring ≥ 0.20, and record a gap reason when insufficient.	P0	Implemented
FR-4.5	The system SHALL detect instruction-like text in retrieved chunks and flag it as suspicious context.	P0	Implemented (5 patterns)
FR-4.6	All retrieval SHALL be filtered by `tenant_id`.	P0	Implemented on both prefetch branches
FR-4.7	The knowledge base SHALL contain ≥ 42 curated markdown documents across 7 categories.	P0	Implemented — exactly 42 documents, 178 indexed chunks
FR-4.8	Embeddings SHALL be computed locally with no external API call.	P0	Implemented (FastEmbed, in-process)

FR-5 — Response drafting

ID	Requirement	Priority	Status
FR-5.1	Drafts SHALL be produced as LLM structured output conforming to `DraftOutput` (`response_text`, `citations`, `tone`, `complexity`, `draft_confidence`).	P0	Implemented
FR-5.2	Every citation carried in `citations` MUST be referenced inline in `response_text` as `[chunk_id]`; citations that are not SHALL be removed rather than trusted.	P0	Implemented as a Pydantic model validator that strips unreferenced citations
FR-5.3	`tone` SHALL be one of `formal`, `friendly`, `apologetic`, `neutral`, `technical`.	P1	Implemented (regex-constrained field)
FR-5.4	`complexity` SHALL be one of `simple`, `moderate`, `complex`.	P1	Implemented
FR-5.5	`draft_confidence` SHALL be recorded for observability and SHALL NEVER be an input to routing.	P0	Implemented — stored as `ConfidenceBreakdown.draft_confidence_logged`, unused by `decide()`
FR-5.6	`response_text` SHALL be between 1 and 4,000 characters.	P1	Implemented
FR-5.7	If every provider fails, the system SHALL emit a safe template draft rather than nothing, and that draft SHALL route to human review.	P0	Implemented

FR-6 — Validation

ID	Requirement	Priority	Status
FR-6.1	The system SHALL run **3** validators on every draft: `pii_leak`, `citations`, `grounding`.	P0	Implemented
FR-6.2	Validators SHALL run concurrently via `asyncio.gather` under a configurable timeout (default 15 s).	P1	Implemented
FR-6.3	A validator that crashes or times out SHALL return `passed=False, blocking=True, errored=True` — fail closed, never fail open.	P0	Implemented
FR-6.4	The grounding validator SHALL compute the fraction of the draft's meaningful words (>3 characters) that appear in the concatenated retrieved context, and compare it against `min_grounding_ratio` (0.80).	P0	Implemented — word overlap, **not** LLM entailment
FR-6.5	The citation validator SHALL confirm that every `[chunk_id]` marker and every structured citation resolves to a retrieved chunk.	P0	Implemented
FR-6.6	The PII validator SHALL scan the **outbound** draft for the same three secret classes as inbound redaction.	P0	Implemented

FR-7 — Safety gates (confidence layer 1)

ID	Requirement	Priority	Status
FR-7.1	The system SHALL evaluate **7** conjunctive veto gates before computing any quality score. Any single failure SHALL route to `HUMAN_REVIEW` regardless of score.	P0	Implemented
FR-7.2	**G1** `G1_policy_critical` — any policy violation of severity `critical` blocks auto-send.	P0	Implemented
FR-7.3	**G2** `G2_pii_leak` — a failing PII validator blocks auto-send.	P0	Implemented
FR-7.4	**G3** `G3_citations_resolve` — unresolvable citations block auto-send.	P0	Implemented
FR-7.5	**G4** `G4_intent_allows_auto` — `complaint` and `unknown` never auto-send.	P0	Implemented via `PREEMPT_INTENTS`
FR-7.6	**G5** `G5_classifier_confident` — classifier confidence below `abstain_threshold` (0.55) blocks auto-send.	P0	Implemented
FR-7.7	**G6** `G6_validators_healthy` — any validator with `errored=True` blocks auto-send.	P0	Implemented
FR-7.8	**G7** `G7_no_suspicious_context` — detected context injection blocks auto-send.	P0	Implemented
FR-7.9	The failing gate's ID SHALL become the decision's `reason_code`, so every escalation has exactly one machine-readable cause.	P0	Implemented

FR-8 — Quality scoring (confidence layer 2)

ID	Requirement	Priority	Status
FR-8.1	The system SHALL compute a weighted score over 5 signals: citation coverage 0.35, grounding entailment 0.25, retrieval relevance 0.20, intent confidence 0.10, tone alignment 0.10.	P0	Implemented; weights sum to **1.00** (verified)
FR-8.2	Quality signals SHALL be compensatory — strength in one may offset weakness in another. Safety signals SHALL NEVER appear among them.	P0	Implemented; no safety term appears in `weights`

ID	Requirement	Priority	Status
FR-8.3	The auto-send threshold SHALL be selected per intent from `config/thresholds.yaml`.	P0	**Not met.** All eight thresholds are configured, but `pipeline/orchestrator.py` applies `thresholds["general"]` (0.70) unconditionally (§15, R-3)
FR-8.4	If all gates pass AND score ≥ threshold → `AUTO_SEND`; otherwise → `HUMAN_REVIEW` with reason code `low_confidence`.	P0	Implemented
FR-8.5	The full breakdown — every signal, the weights, the score, the threshold, and every gate result — SHALL be persisted with the trace.	P0	Implemented

FR-9 — Human review

ID	Requirement	Priority	Status
FR-9.1	Messages routed to `HUMAN_REVIEW` SHALL be persisted to a review queue with an SLA deadline.	P0	Implemented; deadline is hardcoded to 4 hours rather than read from `sla_minutes` (§15, R-4)
FR-9.2	Operators SHALL be able to approve, edit-and-approve, reject, or escalate a review item.	P0	Implemented — all four in `services/review/service.py` and the dashboard; the REST API exposes the first three
FR-9.3	Edits SHALL record an `EditRecord` with the original draft, edited draft, editor, timestamp, reason, and character diff.	P1	Implemented
FR-9.4	State transitions SHALL use optimistic concurrency (`UPDATE … WHERE state = :expected`, `rowcount == 1` wins).	P0	Implemented
FR-9.5	Approved and edited items SHALL be enqueued to the outbox for delivery.	P0	Implemented
FR-9.6	The review item SHALL carry the original customer message and sender address so the operator has full context and the reply is addressable.	P0	Implemented (`inbound_body`, `sender_email`; migrations `019f6dc5d4f0`, `d300852af25a`)

FR-10 — Outbound delivery

ID	Requirement	Priority	Status
FR-10.1	Outbound replies SHALL pass through a transactional outbox with states `QUEUED → SENDING → SENT / FAILED`.	P0	Implemented
FR-10.2	Each outbound message SHALL carry a UNIQUE `idempotency_key`; enqueueing a duplicate key SHALL return `False` rather than creating a second row.	P0	Implemented and enforced by a DB UNIQUE constraint
FR-10.3	A background worker SHALL poll the outbox, claim messages atomically, deliver over Gmail SMTP, and record the outcome.	P0	Implemented (`scripts/start_worker.py`)
FR-10.4	Failed deliveries SHALL be retried up to `max_attempts` and then moved to `DEAD`.	P1	**Deferred.** `OutboxState.DEAD` is defined; no retry counter, no requeue, no DEAD transition (§15, R-5)
FR-10.5	A crashed worker's claimed messages SHALL be reclaimable after a lease expires.	P1	**Deferred.** `claim_lease_minutes` is configured; no lease column exists

FR-11 — Alerting

ID	Requirement	Priority	Status
FR-11.1	The system SHALL dispatch an alert on every routing to human review.	P0	Implemented (called from the orchestrator, exception-isolated)
FR-11.2	Supported alert channels SHALL be: **Gmail SMTP email, Telegram Bot API, and a generic HTTP webhook.**	P0	Implemented
FR-11.3	Alerts SHALL include review ID, trace ID, tenant ID, customer address, the customer's message, the routing action, reason, reason code, score against threshold, the draft, and the citations.	P1	Implemented
FR-11.4	Each channel SHALL fail independently — a Telegram outage SHALL NOT prevent the email alert, and no alert failure SHALL affect the routing decision.	P1	Implemented; each sender returns a boolean and swallows its own exceptions, and the dispatch call itself is wrapped
FR-11.5	Alert content that is interpolated into markup SHALL be escaped.	P1	Implemented (`html.escape` on all Telegram fields)

FR-12 — Observability

ID	Requirement	Priority	Status
FR-12.1	Every processed message SHALL produce a `Trace` capturing intent, confidence, retrieval, draft, validators, confidence breakdown, decision, and all LLM calls.	P0	Implemented
FR-12.2	Each LLM call SHALL record purpose, provider, model, prompt tokens, completion tokens, USD cost, latency, attempt count, and whether failover occurred.	P0	Implemented
FR-12.3	Each pipeline stage SHALL emit a timed trace event.	P1	Implemented (`TraceEventRow`; payloads are currently empty)
FR-12.4	Traces SHALL be queryable via `GET /traces` and `GET /traces/{trace_id}`.	P0	Implemented
FR-12.5	The system SHALL expose operational metrics: total traces, action breakdown, auto-send rate, human-review rate, and total cost.	P1	Implemented (`GET /metrics`)

FR-13 — Policy enforcement

ID	Requirement	Priority	Status
FR-13.1	The system SHALL enforce **10** policy rules defined in `config/policies.yaml` with no code change required to add or tune one.	P0	Implemented
FR-13.2	Rules SHALL support four detection types: `amount_over`, `regex_any`, `regex_absent`, `price_mismatch`.	P0	Implemented
FR-13.3	Rules SHALL be able to target inbound text or outbound draft text, and to apply only for named intents.	P0	Implemented (`applies_to`, `applies_when.intent`)
FR-13.4	Critical violations SHALL block auto-send via Gate G1.	P0	Implemented
FR-13.5	Warning violations SHALL be recorded on the trace without blocking.	P0	Implemented as non-blocking. Note: the v1 draft claimed warnings reduce the tone score; they do not — `tone_alignment` is currently a constant 1.0

8. Non-Functional Requirements

NFR-1 — Performance

ID	Requirement	Target	Evidence / status
NFR-1.1	End-to-end pipeline latency, cloud path	< 10 s (P95)	Per-stage timings recorded per trace; percentile aggregation not yet implemented
NFR-1.2	End-to-end pipeline latency, local-only path	< 30 s (P95)	Same; Ollama timeout is configured at 120 s
NFR-1.3	Hybrid vector search latency	< 200 ms	`RetrievalResult.latency_ms` recorded per message
NFR-1.4	Full test-suite wall time	< 60 s	**38.32 s** measured for 68 tests
NFR-1.5	LLM calls per message	≤ 2 in the normal path	By construction: one classification call, one drafting call. Grounding is deterministic (ADR-003 revised)

NFR-2 — Reliability

ID	Requirement	Target	Evidence / status
NFR-2.1	LLM provider failover	Automatic, priority-ordered	`OpenRouter → Claude → Ollama` for drafting; verified by `tests/unit/test_providers.py::test_router_failover`
NFR-2.2	Graceful degradation to no network	Full pipeline operates in `local_only`	Priority chain plus keyword and template fallbacks
NFR-2.3	Exactly-once outbound delivery	No duplicate send for a given idempotency key	DB UNIQUE constraint; `test_outbox_no_double_send`
NFR-2.4	Fail-closed validation	Crashed or timed-out validator blocks auto-send	`test_validator_error_routes_to_human`
NFR-2.5	Alert isolation	An alert failure never changes a routing decision	Dispatch is exception-wrapped at the call site
NFR-2.6	Classification failover across providers	3-deep	**Not met** — only OpenRouter defines a `classification` model; the chain is OpenRouter → keyword fallback (§15, R-8)

NFR-3 — Security

ID	Requirement	Target	Evidence / status
NFR-3.1	Secrets and PII never enter an LLM prompt	0 occurrences	Met for classification; **not met for drafting** (§15, R-1)
NFR-3.2	No secret or PII leaves in an outbound reply	Gate G2 blocks	`test_pii_validator_detect_cc`, `test_pii_validator_detect_secret_key`
NFR-3.3	Prompt-injection content in the corpus cannot influence a send	Gate G7 blocks	`test_injection_detector_positive`
NFR-3.4	No internal infrastructure detail in a reply	Critical policy rule `no_security_details`	`config/policies.yaml`
NFR-3.5	Credentials live in the environment, never in source	0 hardcoded model IDs or keys in `src/`	`scripts/check_no_hardcoded_models.py`, enforced by pre-commit and `test_no_hardcoded_models`
NFR-3.6	Tenant isolation on retrieval	Every query filtered by `tenant_id`	`adapters/vector/qdrant_store.py` — filter on both prefetch branches

ID	Requirement	Target	Evidence / status
NFR-3.7	No credential is committed to the repository	0	**Not met** — `.env.example` is tracked and contains a live-looking Gmail app password. Must be rotated and scrubbed

NFR-4 — Maintainability

ID	Requirement	Target	Evidence / status
NFR-4.1	`domain/` and `services/` never import an adapter or a vendor SDK	Contract enforced in CI	`uv run lint-imports` — 1 contract kept, 0 broken, over 102 files / 222 dependencies
NFR-4.2	Behaviour is configuration-driven	Thresholds, weights, policies, prompts, model IDs in YAML	`config/*.yaml`; changing a threshold and re-running flips a decision with no restart
NFR-4.3	Database portability	SQLite → PostgreSQL is a URL change	Guaranteed by NFR-4.1 plus repository Protocols; not yet exercised
NFR-4.4	Automated test coverage	Unit + pipeline + integration green	**68/68 passing.** `tests/e2e/` is empty
NFR-4.5	Static typing in the protected layers	`disallow_untyped_defs` on `domain`, `services`, `ports`	Configured in `pyproject.toml`; 9 mypy errors remain, concentrated in adapters and one in `domain/models.py`
NFR-4.6	Lint cleanliness	0 ruff findings	**Not met** — 24 findings (21 auto-fixable), none in `services/confidence/`

NFR-5 — Operational

ID	Requirement	Target	Evidence / status
NFR-5.1	Zero container dependency	Runs on Python 3.12 alone	Embedded Qdrant + SQLite; verified on Python 3.12.13
NFR-5.2	Model spend ceiling	≤ $2.00/day, alert at 80%	**Configured, not enforced.** The `budgets` block in `providers.yaml` is read by no code. Per-call cost *is* recorded and aggregated
NFR-5.3	Per-intent SLA deadlines	default 120 min; complaint/refund 60 min	**Configured, not applied.** The orchestrator hardcodes a 4-hour deadline
NFR-5.4	Schema evolution	Versioned migrations	Alembic, 3 revisions, head `d300852af25a`
NFR-5.5	Concurrent DB access	No writer starvation at demo volume	SQLite WAL, `busy_timeout=5000`, FK enforcement; asserted by `test_engine_wal_mode`

---

9. Constraints & Assumptions

9.1 Constraints

Constraint	Consequence for the design
Solo developer, ~4 weeks	Vertical slices; ruthless scope control; every deferral recorded rather than hidden
Free tier only	Embedded Qdrant, SQLite, local Ollama, Gmail free tier, Telegram Bot API

Constraint	Consequence for the design
LLM API keys are temporary and demo-scoped	A local fallback is mandatory, not optional (ADR-002)
No Docker experience available	Zero containers; the whole runtime is `uv run` (ADR-007)
Must be explained live, component by component	Explicit async pipeline instead of an orchestration framework (ADR-005)
The live demo must not fail	`local_only` mode; embedded stores; no network-dependent critical path
Python 3.12.x	`StrEnum`, PEP 695 generics (`async def _time_stage[T]`), `X | None` unions throughout
Safety is non-negotiable	Gates precede scoring; no safety signal may ever become a weight (ADR-001)

9.2 Assumptions

1. A curated knowledge base of ≥ 40 markdown articles covering standard support scenarios exists and is re-ingested when it changes.
2. Either an LLM API key (OpenRouter and/or Anthropic) is available, **or** a machine with enough RAM to run the configured local model (`qwen3:8b`) is available.
3. Gmail credentials with an app password are configured for inbound polling and outbound delivery.
4. Human operators are available to action escalations within the SLA window.
5. All traffic is English.
6. All data is synthetic; no real customer PII passes through the system.

---

10. Data Requirements

10.1 Knowledge base

Property	Value
Location	`data/knowledge_base/`
Format	Markdown with YAML frontmatter
Documents	**42**
Categories	billing (9), pricing (7), cancellations (6), refunds (6), enterprise (5), invoices (5), troubleshooting (4)
Chunking	Heading-aware; target 450 tokens (~338 words), 60-token overlap; deterministic `{doc_id}:{ordinal}` chunk IDs
Indexed chunks	**178** points in Qdrant collection `taskflow_kb`
Dense embedding	`BAAI/bge-small-en-v1.5`, 384 dimensions, cosine
Sparse embedding	`Qdrant/bm25` with IDF modifier
Storage	Embedded Qdrant at `./data/qdrant` — no server, no container

10.2 Evaluation datasets

All synthetic, in `data/datasets/`.

Dataset	Records	Purpose
`classifier_train.jsonl`	35	Labelled intent training split (reserved for the deferred trained classifier)

Dataset	Records	Purpose
`classifier_validation.jsonl`	14	Validation split
`classifier_test.jsonl`	14	Held-out test split
`golden_eval.jsonl`	70	Hand-checked end-to-end quality set
`adversarial.jsonl`	127	Injection attempts, policy traps, edge cases
`tier-1.jsonl` … `tier-4.jsonl`	126 / 126 / 57 / 42	Tiered difficulty corpora
`demo_scenarios.jsonl`	5	Scripted presentation scenarios

Leakage between splits is checkable with `scripts/check_leakage.py`.

10.3 Persistent state

Store	Technology	Contents
Relational	SQLite in WAL mode via `aiosqlite` + async SQLAlchemy	9 tables: `inbox`, `traces`, `trace_events`, `reviews`, `edit_records`, `outbox`, `llm_calls`, `audit`, `alerts`
Vector	Embedded Qdrant	178 knowledge-base chunks with dense + sparse vectors and payload metadata

Every table carries a `tenant_id` column. Two UNIQUE constraints are safety-critical and directly tested: `inbox.dedupe_key` and `outbox.idempotency_key`.

---

11. Integration Points

System	Protocol	Direction	Purpose	Status
OpenRouter API	HTTPS (httpx)	Outbound	Primary LLM — `openai/gpt-4o-mini` for all purposes	Active
Anthropic Claude API	HTTPS (`anthropic` SDK)	Outbound	Secondary LLM — `claude-haiku-4-5` for draft/grounding, `claude-sonnet-4-5` for data generation	Active for drafting; pricing entries are placeholder zeros
Ollama	HTTP, `localhost:11434`	Outbound	Local fallback LLM — `qwen3:8b`	Active; the whole of `local_only` mode
Gmail IMAP	IMAP over TLS, `imap.gmail.com:993`	Inbound	Poll unseen support email	Active
Gmail SMTP	SMTP+STARTTLS, `smtp.gmail.com:587`	Outbound	Deliver replies and email alerts	Active
Telegram Bot API	HTTPS	Outbound	Push escalation alerts to an operator chat	Active
Generic HTTP webhook	HTTPS POST	Outbound	Integration hook — receives the full review item as JSON	Active

---

12. Acceptance Criteria

Checked only where a named test or command proves it. Unchecked items are honest gaps.

12.1 Core pipeline

- ✅ A message is ingested, classified, retrieved for, drafted, validated, gated, scored, and routed, end to end — verified by `tests/integration/test_pipeline_e2e.py::test_full_pipeline_auto_send_flow`.
- ✅ A clean draft with all gates passing and a score above threshold is auto-sent and enqueued to the outbox — verified by `tests/pipeline/test_orchestrator.py::test_clean_draft_auto_sends`.
- ✅ A complaint is drafted but blocked from auto-send by the intent gate and routed to review — verified by `tests/pipeline/test_orchestrator.py::test_complaint_escalates`.
- ✅ Retrieval combines dense and sparse search with RRF and returns tenant-filtered chunks — verified by `tests/unit/test_retrieval.py::test_retrieve_context_service`.
- ✅ When the LLM path fails, classification degrades to the keyword classifier instead of failing the message — verified by `tests/unit/test_classifier.py::test_fallback_on_all_providers_failed`.
- ✅ When every provider fails, drafting produces a safe template rather than an exception — verified by `tests/unit/test_drafting.py::test_generate_draft` together with `services/draft/service.py` fallback path.
- ▫ A credit-card number in an inbound message is redacted before **every** LLM call *(not yet verified — redaction is applied to the classification call only; the drafting call receives the unredacted body).*
- ▫ The full local-only path runs end to end with no network *(not yet verified — no automated test asserts `local_only` behaviour).*

12.2 Safety

- ✅ A draft containing a $750 refund promise triggers Gate G1 and routes to human review **even with every quality signal at 1.0 and a score above 0.95** — verified by `tests/unit/test_gates.py::test_critical_policy_violation_never_auto_sends`.
- ✅ The `refund_ceiling` rule fires on a draft promising more than $500 — verified by `tests/unit/test_policy.py::test_refund_ceiling_violation`.
- ✅ A draft containing a card number or secret key fails the PII validator and Gate G2 blocks the send — verified by `tests/unit/test_gates.py::test_pii_in_draft_blocks_send`, `tests/unit/test_drafting.py::test_pii_validator_detect_cc`, `::test_pii_validator_detect_secret_key`.
- ✅ A citation that resolves to no retrieved chunk blocks the send at Gate G3 — verified by `tests/unit/test_gates.py::test_unresolvable_citation_blocks_send` and `tests/unit/test_drafting.py::test_citations_validator_fail`.
- ✅ Classifier confidence below 0.55 blocks the send at Gate G5 — verified by `tests/unit/test_gates.py::test_classifier_abstention_blocks_send`.
- ✅ A validator that errors blocks the send at Gate G6 rather than being skipped — verified by `tests/unit/test_gates.py::test_validator_error_routes_to_human`.
- ✅ A retrieved chunk containing "ignore previous instructions" is flagged as suspicious context — verified by `tests/unit/test_retrieval.py::test_injection_detector_positive`, with the negative control at `::test_injection_detector_negative`.
- ✅ A legal threat in an inbound message produces a critical violation — verified by `tests/unit/test_policy.py::test_legal_threat_inbound_violation`.
- ✅ A request for the customer's password produces a critical violation — verified by `tests/unit/test_policy.py::test_no_credential_request_violation`.
- ✅ A data-deletion request routes to the privacy path — verified by `tests/unit/test_policy.py::test_gdpr_deletion_routing`.
- ✅ A clean draft produces no violations — verified by `tests/unit/test_policy.py::test_clean_draft_no_violations`.
- ✅ The control case holds: no gate failure and a high score does auto-send — verified by `tests/unit/test_gates.py::test_clean_draft_auto_sends`.

- ▫ Gate G7 blocks an auto-send end to end when injection is detected mid-pipeline *(not yet verified — the detector is tested in isolation; no pipeline-level test drives a poisoned chunk through to a routing decision).*

12.3 Human review and delivery

- ✅ Escalated items appear on the pending queue in SLA-deadline order — verified by `tests/unit/test_review_worker.py::test_list_pending_reviews` and `tests/unit/test_repositories.py::test_review_pending_list`.
- ✅ An operator can approve an item, and the approved reply is enqueued to the outbox — verified by `tests/unit/test_review_worker.py::test_approve_review`.
- ✅ An operator can edit and approve, and the edit is recorded with a character diff — verified by `tests/unit/test_review_worker.py::test_edit_and_approve_review`.
- ✅ Two operators actioning the same item simultaneously: exactly one wins — verified by `tests/unit/test_review_worker.py::test_optimistic_lock_collision` and `tests/unit/test_repositories.py::test_review_transition_optimistic_lock`.
- ✅ The same reply is never enqueued twice for one idempotency key — verified by `tests/unit/test_repositories.py::test_outbox_enqueue_idempotent` and `::test_outbox_no_double_send`.
- ✅ The worker claims a batch atomically and marks it sent — verified by `tests/unit/test_review_worker.py::test_worker_process_batch` and `tests/unit/test_repositories.py::test_outbox_claim_and_mark`.
- ✅ The review and approval path works over REST — verified by `tests/unit/test_api.py::test_list_and_approve_review_api`.
- ▫ A delivery failure is retried and eventually moved to `DEAD` *(not yet verified — retry and DEAD transitions are not implemented;* `deliver_message()` *currently returns* `True` *on its fallthrough path, so a failed SMTP send is still marked* `SENT`*).*
- ▫ A crashed worker's claimed messages are reclaimed after the lease expires *(not yet verified — no lease column exists).*

12.4 Observability and operations

- ✅ Every processed message has a retrievable trace — verified by `tests/unit/test_repositories.py::test_trace_start_and_get` and `tests/unit/test_api.py::test_list_traces_api`.
- ✅ Each pipeline stage writes a timed trace event — verified by `tests/unit/test_repositories.py::test_trace_event_written`.
- ✅ The trace is finalised with its routing decision — verified by `tests/unit/test_repositories.py::test_trace_finish`.
- ✅ Per-call LLM cost is computed from configured pricing — verified by `tests/unit/test_providers.py::test_cost_calculation`.
- ✅ Metrics expose total traces, action breakdown, auto-send rate, human-review rate, and total cost — verified by `tests/unit/test_api.py::test_metrics_api`.
- ✅ The API is healthy and ingests a message end to end — verified by `tests/unit/test_api.py::test_healthz_endpoint`, `::test_ingest_message_api`.
- ✅ All nine tables are created by the schema — verified by `tests/unit/test_repositories.py::test_all_tables_created`.
- ✅ The database runs in WAL mode — verified by `tests/unit/test_repositories.py::test_engine_wal_mode`.
- ▫ Latency is reported as a P95 percentile *(not yet verified — per-stage timings are recorded but not aggregated).*
- ▫ Daily spend is capped and an alert fires at 80% of budget *(not yet verified — the* `budgets` *block is not read by any code).*

12.5 Architecture and quality

- ☑ `domain/` and `services/` import no adapter and no vendor SDK — verified by `uv run lint-imports` (1 contract kept, 0 broken) and `tests/unit/test_imports.py::test_package_imports`.
- ☑ No model identifier is hardcoded anywhere in `src/` — verified by `tests/unit/test_imports.py::test_no_hardcoded_models` and `scripts/check_no_hardcoded_models.py`.
- ☑ Alert channels dispatch independently and degrade silently when unconfigured — verified by `tests/unit/test_alert.py::test_dispatch_alert`, `::test_send_email_alert_unconfigured`, `::test_send_telegram_alert_unconfigured`.
- ☑ Provider failover selects the next provider on a transient failure and raises `AllProvidersFailed` only when all are exhausted — verified by `tests/unit/test_providers.py::test_router_primary_success`, `::test_router_failover`, `::test_router_all_failed`.
- ☑ The quality weights sum to exactly 1.0 — verified by inspection of `config/thresholds.yaml` (0.35 + 0.25 + 0.20 + 0.10 + 0.10 = 1.00).
- ☐ The repository is lint-clean *(not yet verified — 24 ruff findings)*.
- ☐ The repository is type-clean *(not yet verified — 9 mypy errors across 5 files)*.
- ☐ End-to-end scenario tests exist *(not yet verified — `tests/e2e/` is empty)*.

---

13. Requirement → Evidence Traceability Matrix

The P0 requirements, each mapped to the code that satisfies it and the command that proves it. This is the table to read alongside a terminal.

Req ID	Description	Satisfied by	Verified by	Status
FR-1.1	REST message ingestion	`src/taskflow/api/app.py` → `POST /messages`	`pytest tests/unit/test_api.py::test_ingest_message_api`	✅ Verified
FR-1.4	Dedupe key persisted under UNIQUE	`adapters/db/orm.py` `InboxRow.dedupe_key`; `repositories.py` `try_claim`	`pytest tests/unit/test_repositories.py::test_inbox_try_claim_unique`	⚠️ Built, not wired into the pipeline
FR-2.1–2.3	PII redaction of cards, SSNs, secrets	`services/preprocess/service.py`	`pytest tests/unit/test_drafting.py::test_pii_validator_detect_cc` (same patterns)	✅ Verified
FR-2.5	Only redacted text reaches the LLM	`pipeline/orchestrator.py` (classification path)	—	❌ Not met — drafting uses `msg.body_text`
FR-3.1	8-intent taxonomy	`domain/enums.py` `Intent`	`pytest tests/unit/test_classifier.py` (4 tests)	✅ Verified
FR-3.2	LLM structured-output classification	`services/classify/service.py`	`pytest tests/unit/test_classifier.py::test_structured_llm_classification`	✅ Verified
FR-3.3	Deterministic keyword fallback	`services/classify/service.py::_fallback_classify`	`pytest tests/unit/test_classifier.py::test_fallback_on_all_providers_failed`	✅ Verified
FR-4.1	Hybrid dense + sparse retrieval	`adapters/vector/qdrant_store.py`	`pytest tests/integration/test_pipeline_e2e.py` (real embedded Qdrant)	✅ Verified

Req ID	Description	Satisfied by	Verified by	Status
FR-4.2	RRF fusion	`qdrant_store.py` → `models.FusionQuery(Fusion.RRF)`	Same integration test	✅ Verified
FR-4.4	Sufficiency evaluation	`services/retrieve/sufficiency.py`; `config/thresholds.yaml` `retrieval`	`pytest tests/unit/test_retrieval.py::test_sufficiency_pass`, `::test_sufficiency_fail_score`	✅ Verified
FR-4.5	Prompt-injection detection in chunks	`services/retrieve/injection.py`	`pytest tests/unit/test_retrieval.py::test_injection_detector_positive`	✅ Verified
FR-4.6	Tenant filtering on retrieval	`qdrant_store.py` — filter on both prefetch branches	Code inspection + integration test	✅ Verified
FR-4.7	42-document knowledge base	`data/knowledge_base/`	`find data/knowledge_base -name '*.md' | wc -l` → 42	✅ Verified
FR-5.1	`DraftOutput` contract	`domain/models.py::DraftOutput`	`pytest tests/unit/test_drafting.py::test_generate_draft`	✅ Verified
FR-5.2	Citations must be referenced inline	`domain/models.py::_citation_markers_present`	`pytest tests/unit/test_drafting.py::test_citations_validator_pass`	✅ Verified
FR-5.5	`draft_confidence` never routes	`services/confidence/scorer.py::decide` — reads only gates, score, threshold	`pytest tests/unit/test_gates.py` (all 6 pass a `draft_confidence` that is ignored)	✅ Verified
FR-5.7	Template fallback on total provider failure	`services/draft/service.py` except-branch	Code inspection; exercised by `test_generate_draft` fallback path	✅ Verified
FR-6.1	Three concurrent validators	`services/validate/runner.py`	`pytest tests/unit/test_drafting.py::test_run_validators_parallel`	✅ Verified
FR-6.3	Fail-closed on validator error	`runner.py::safe_run`	`pytest tests/unit/test_gates.py::test_validator_error_routes_to_human`	✅ Verified
FR-6.4	Grounding ratio ≥ 0.80	`services/validate/grounding.py`; `thresholds.yaml` `gates.min_grounding_ratio`	`pytest tests/unit/test_drafting.py::test_grounding_validator_pass`	✅ Verified
FR-6.5	Citation resolution	`services/validate/citations.py`	`pytest tests/unit/test_drafting.py::test_citations_validator_fail`	✅ Verified
FR-7.1	Seven conjunctive gates before scoring	`services/confidence/gates.py::evaluate_gates`	`pytest tests/unit/test_gates.py` (6 tests, all green)	✅ Verified
FR-7.2	G1 vetoes critical policy violations	`gates.py` G1 branch; `services/policy/service.py`	`pytest tests/unit/test_gates.py::test_critical_policy_violation_never_auto_sends`	✅ Verified

Req ID	Description	Satisfied by	Verified by	Status
FR-7.5	G4 preempts `complaint` / `unknown`	`domain/enums.py::PREEMPT_INTENTS`	`pytest tests/pipeline/test_orchestrator.py::test_complaint_escalates`	✅ Verified
FR-7.9	Failing gate ID becomes the reason code	`scorer.py::decide`	All six `test_gates.py` assertions check `reason_code`	✅ Verified
FR-8.1	Five weighted signals summing to 1.0	`config/thresholds.yaml` `weights`; `scorer.py::compute`	Arithmetic check: 0.35+0.25+0.20+0.10+0.10 = 1.00	✅ Verified
FR-8.3	Per-intent auto-send thresholds	`config/thresholds.yaml` `thresholds`	—	❌ Not met — orchestrator applies `general` (0.70) only
FR-8.4	Gate-then-score decision logic	`scorer.py::decide`	`pytest tests/unit/test_gates.py::test_clean_draft_auto_sends`	✅ Verified
FR-9.1	Review queue with SLA deadline	`pipeline/orchestrator.py`; `orm.py::ReviewRow`	`pytest tests/unit/test_repositories.py::test_review_create_and_get`	⚠️ Deadline hardcoded to 4 h
FR-9.2	Approve / edit / reject / escalate	`services/review/service.py`	`pytest tests/unit/test_review_worker.py` (5 tests)	✅ Verified
FR-9.4	Optimistic concurrency on transitions	`repositories.py::SQLiteReviewRepository.transition`	`pytest tests/unit/test_review_worker.py::test_optimistic_lock_collision`	✅ Verified
FR-10.1	Transactional outbox state machine	`orm.py::OutboxRow`; `repositories.py::SQLiteOutboxRepository`	`pytest tests/unit/test_repositories.py::test_outbox_claim_and_mark`	✅ Verified
FR-10.2	UNIQUE idempotency key	`orm.py` `uq_outbox_idempotency_key`	`pytest tests/unit/test_repositories.py::test_outbox_no_double_send`	✅ Verified
FR-10.3	Background delivery worker	`worker/outbox_worker.py`; `scripts/start_worker.py`	`pytest tests/unit/test_review_worker.py::test_worker_process_batch`	✅ Verified
FR-11.1–11.2	Multi-channel escalation alerting	`services/alert/service.py`	`pytest tests/unit/test_alert.py` (4 tests)	✅ Verified
FR-11.4	Independent channel failure	Each sender returns `bool` and swallows its own exception	`pytest tests/unit/test_alert.py::test_dispatch_alert`	✅ Verified
FR-12.1	Full per-message trace	`domain/models.py::Trace`; `repositories.py::SQLiteTraceRepository`	`pytest tests/unit/test_repositories.py::test_trace_start_and_get`	✅ Verified
FR-12.2	Per-call LLM cost accounting	`domain/models.py::LLMCall`; `services/cost/service.py`	`pytest tests/unit/test_providers.py::test_cost_calculation`	✅ Verified

Req ID	Description	Satisfied by	Verified by	Status
FR-12.4	Trace query API	`api/app.py` → `GET /traces`, `GET /traces/{id}`	`pytest tests/unit/test_api.py::test_list_traces_api`	✅ Verified
FR-13.1	Ten configurable policy rules	`config/policies.yaml`; `services/policy/service.py`	`pytest tests/unit/test_policy.py` (5 tests)	✅ Verified
NFR-2.1	Provider failover chain	`adapters/llm/router.py`	`pytest tests/unit/test_providers.py::test_router_failover`, `::test_router_all_failed`	✅ Verified
NFR-3.5	No hardcoded model IDs or keys in source	`scripts/check_no_hardcoded_models.py` + pre-commit	`pytest tests/unit/test_imports.py::test_no_hardcoded_models`	✅ Verified
NFR-4.1	Hexagonal import boundary	`pyproject.toml [tool.importlinter]`	`uv run lint-imports` → 1 kept, 0 broken	✅ Verified
NFR-4.4	Automated test coverage	`tests/`	`uv run pytest -q` → 68 passed in 38.32 s	✅ Verified
NFR-5.1	No container dependency	Embedded Qdrant + SQLite	Whole suite runs on `uv run` with Python 3.12.13	✅ Verified
NFR-5.2	Daily spend cap enforced	`config/providers.yaml budgets`	—	❌ Not met — config is read by no code
NFR-5.4	Versioned schema migrations	`adapters/db/migrations/`	`uv run alembic heads` → `d300852af25a (head)`	✅ Verified

Legend: ✅ code exists and a named test or command proves it · ⚠️ code exists but is partially wired or partially applied · ❌ specified, not met today.

14. Risks & Mitigations

#	Risk	Likelihood	Impact	Mitigation in place	Residual action
R-1	**Unredacted customer text reaches the drafting model.** `services/draft/service.py` interpolates `msg.body_text`, not `msg.body_redacted`.	High (occurs on every drafted message)	High — defeats NFR-3.1 for the drafting call	Outbound PII validator and Gate G2 still catch a leak in the *reply*; inbound redaction protects the classification call	**Fix:** pass `msg.body_redacted` in the drafting prompt and add a test asserting the prompt contains no raw secret
R-2	**Classifier prompt lists intents that do not exist** (`cancellation`, `enterprise`), so those labels fail `Intent` validation and silently degrade to the keyword classifier.	High	Medium — degraded classification quality with no visible error	Keyword fallback keeps the message flowing; Gate G5 catches low confidence	**Fix:** generate the prompt's intent list from the `Intent` enum

#	Risk	Likelihood	Impact	Mitigation in place	Residual action
R-3	**Per-intent thresholds are configured but not applied.** Every intent is scored against `general` = 0.70.	Certain	Medium-High — `refund` (0.90) and `billing` (0.80) auto-send at a materially lower bar than designed	Gates still veto every unsafe case independently of the threshold; no *unsafe* send results	**Fix:** select the threshold by `state.intent.value` with a `general` default
R-4	**SLA deadlines are hardcoded to 4 hours** and no sweeper checks for breaches.	Certain	Low-Medium	Alerts fire immediately on escalation, so nothing waits silently	**Fix:** read `sla_minutes`; add a periodic `due()` sweep
R-5	**A failed SMTP delivery is still marked `SENT`.** `deliver_message()` returns `True` on its fallthrough path, and there is no retry, lease, or `DEAD` transition.	Medium	High — a customer reply can be silently lost	Idempotency key prevents *duplicates*; the outbox row persists the body for manual recovery	**Fix:** return the transport result; implement retry counting and the `DEAD` terminal state
R-6	~~The Gmail poller raised `AttributeError` on its first message — `settings().tenant_id` did not exist.~~ **Resolved 2026-08-24**: the poller now reads `settings().taskflow_tenant_id`.	—	—	—	Add a smoke test for the poller's message construction so this cannot regress
R-7	**The Telegram channel connector is dead code** — `adapters/channels/telegram.py` fails to import (`DeliveryReceipt` is not in `domain.models`; `Channel.TELEGRAM` does not exist).	Certain if imported	Low — nothing imports it; Telegram *alerting* works through `services/alert`	Alerting is independent of the connector	**Fix:** repair or delete; do not leave a non-importing module in `adapters/`
R-8	**Classification cannot fail over.** Only OpenRouter defines a `classification` model, so `model_for("claude", "classification")` raises `KeyError`, which the router does not catch as a provider error.	High when OpenRouter is unavailable	Medium	The keyword classifier absorbs the failure, so the message still flows	**Fix:** add `classification` entries for Claude and Ollama, and catch `KeyError` in the router loop
R-9	**A live-looking Gmail app password is committed** in the git-tracked `.env.example`.	Already occurred	High	`.gitignore` covers `.env`; pre-commit runs `detect-private-key` (which does not catch app passwords)	**Act now:** rotate the credential, scrub the file, and consider history rewrite
R-10	**Claude pricing is `0.0 / 0.0` placeholder**, so Claude usage records as $0 and the cost ledger understates spend whenever failover engages.	Medium	Medium	OpenRouter (the primary) is priced correctly, so the common path is accurate	**Fix:** populate real pricing in `config/providers.yaml`

#	Risk	Likelihood	Impact	Mitigation in place	Residual action
R-11	LLM hallucination produces a fluent but unsupported reply.	Medium	High	Sufficiency check; grounding ratio; citation resolution against real chunk IDs; Gate G3; human review fallback	Grounding is word-overlap, which is a proxy — an entailment check is the natural upgrade
R-12	Knowledge-base staleness — the index diverges from the corpus.	Medium	Medium	Deterministic chunk IDs make re-ingestion idempotent	Operational discipline: re-run `scripts/ingest_kb.py` after any KB edit
R-13	Lint and type findings accumulate in the adapter layer.	Occurring	Low-Medium	The protected `domain` / `services` layers are clean, and the boundary is CI-enforced	Run `uv run ruff check --fix .` (21 of 24 auto-fixable) and resolve the 9 mypy errors
R-14	No end-to-end scenario coverage — `tests/e2e/` is empty.	Certain	Medium	Integration tests exercise the real Qdrant and SQLite path	Build the five demo scenarios as e2e tests

15. Known Defects Register

Cross-referenced from §14. Listed separately because a reviewer will look for exactly this list, and its absence is more damaging than its contents.

Ref	Defect	Location	Severity
R-1	Unredacted body sent to the drafting model	`services/draft/service.py`	High
R-2	Classifier prompt enumerates two non-existent intents	`services/classify/service.py`	High
R-3	Per-intent thresholds configured but not applied	`pipeline/orchestrator.py`	High
R-4	SLA deadline hardcoded; no breach sweeper	`pipeline/orchestrator.py`	Medium
R-5	Failed delivery marked `SENT`; no retry or `DEAD`	`worker/outbox_worker.py`	High
R-7	Telegram connector does not import	`adapters/channels/telegram.py`	Low
R-8	Classification has no provider failover	`config/providers.yaml`, `adapters/llm/router.py`	Medium
R-9	Credential committed in `.env.example`	`.env.example`	High
R-10	Claude pricing placeholder zeros	`config/providers.yaml`	Medium
—	`budgets` block read by no code	`config/providers.yaml`	Medium
—	`ports/vector_store.py` imports from `services/`	`src/taskflow/ports/vector_store.py`	Low
—	Makefile targets reference files that do not exist (`generate_kb.py`, `preflight.py`, `taskflow.ml.*`, `taskflow.api.main`, `demo_seed.db`)	`Makefile`	Low

16. Glossary

Term	Definition
Abstain threshold	The classifier-confidence floor (0.55) below which Gate G5 blocks auto-send, on the principle that an unsure classification should not drive an autonomous action
Auto-send	A routing decision in which the drafted reply is enqueued for delivery with no human reading it
Compensatory score	A weighted sum in which strength in one signal can offset weakness in another — appropriate for quality, never for safety
Conjunctive gate	A binary safety check acting as a veto: any single failure blocks auto-send regardless of every other signal
Dedupe key	The identifier under a UNIQUE constraint that makes reprocessing the same inbound message impossible
Fail-closed	A validator that crashes or times out returns *failed and blocking*, never *passed by default*
Grounding ratio	The fraction of a draft's meaningful words (>3 characters) that also appear in the retrieved context — a word-overlap proxy for factual support
Idempotency key	The UNIQUE outbox value that guarantees a given reply is delivered at most once
Local-only mode	`TASKFLOW_LLM_MODE=local_only` — the provider chain collapses to Ollama, and the system runs with no network at all
Optimistic concurrency	`UPDATE … WHERE state = :expected`; `rowcount == 1` means this actor won the race
Preempt intent	An intent (`complaint`, `unknown`) that Gate G4 makes permanently ineligible for auto-send
Reason code	The single machine-readable cause attached to every routing decision — a gate ID, `auto_send`, or `low_confidence`
RRF	Reciprocal Rank Fusion — merges the dense and sparse ranked lists by reciprocal rank rather than by incomparable raw scores
Sufficiency	The judgement that retrieval returned enough evidence (top score ≥ 0.35, ≥ 2 chunks ≥ 0.20) to justify drafting an answer at all
Trace	The complete, persisted record of one message's journey — every stage, decision, validator, gate, weight, score, and model call
Transactional outbox	Persist-then-deliver: the reply is committed to the database before any send is attempted, so a crash loses nothing
Two-layer confidence	ADR-001: conjunctive safety gates first, weighted quality score second — never one blended number
Veto	A safety outcome that cannot be outvoted by any quantity of good scores elsewhere

Companion document: `PRODUCT_SPECIFICATION.md` — the technical specification of how each requirement here is implemented. Architectural rationale is in `02_ARCHITECTURE_DECISIONS.md`.